Mock Assessment - 1 (ArrayList) Without SEB

Java solutions (answer key) | all sample and hidden test cases verified

Q1. Check Whether an Element Exists in an ArrayList (50 marks)

Build the ArrayList with the five names, read the search string, and use the built-in **contains()** method. It is case-sensitive, so "jessy" and "priya" print *Element not found.*

Java Code (Main.java)

```
import java.util.ArrayList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String target = sc.nextLine();

        ArrayList<String> names = new ArrayList<>();
        names.add("Asha");
        names.add("Rahul");
        names.add("Vicky");
        names.add("Jessy");
        names.add("Priya");

        // Built-in method: contains() (case-sensitive)
        if (names.contains(target)) {
            System.out.println("Element found.");
        } else {
            System.out.println("Element not found.");
        }
    }
}
```

Test cases checked

Input	Expected output
Jessy	Element found.
Rohan	Element not found.
jessy	Element not found.
Rahul	Element found.
Vicky	Element found.
priya	Element not found.

Time complexity: O(n) for contains(), n = 5.

Q2. Double the Odd Numbers in an ArrayList (50 marks)

Read the space-separated integers into an ArrayList. Loop over indices; if **v % 2 != 0** (this also works for negative odd numbers, where v % 2 is -1), replace the element in place using **set(i, v * 2)**. Print the list with a StringBuilder for speed (N up to 10^5).

Java Code (Main.java)

```
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.util.ArrayList;
import java.util.StringTokenizer;

public class Main {
    public static void main(String[] args) throws Exception {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StringTokenizer st = new StringTokenizer(br.readLine());
```

```
        ArrayList<Integer> list = new ArrayList<>();
        while (st.hasMoreTokens()) {
            list.add(Integer.parseInt(st.nextToken()));
        }

        // Update in place: odd numbers (also negative) are doubled (max 2*10^9 fits in int).
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < list.size(); i++) {
            int v = list.get(i);
            if (v % 2 != 0) {
                list.set(i, v * 2);
            }
        }
        for (int i = 0; i < list.size(); i++) {
            if (i > 0) sb.append(' ');
            sb.append(list.get(i));
        }
        System.out.println(sb);
    }
}
```

Test cases checked

Input	Expected output
1 3 5 7	2 6 10 14
1 2 3 4 5	2 2 6 4 10
1 2 3 6 5 10 7 14	2 2 6 6 10 10 14 14
2 4 6 8 10	2 4 6 8 10
10 15 20 25 30 35 40	10 30 20 50 30 70 40

Time complexity: O(N). The largest result is 2 x 10^9 = 2,000,000,000, which still fits in an int (max 2,147,483,647).